

Windows приложения.
Компоненты формы.

Многооконные приложения

Windows приложения, как правило, состоят из нескольких взаимодействующих между собой окон. В этом разделе мы познакомимся с техникой создания многооконных приложений в Window Forms.

Для того, чтобы добавить в проект форму, нужно войти в меню Проект и выбрать пункт Добавить форму. В выпадающем списке выбрать пункт Форма. После этого в проекте появится новая форма.

Многооконные приложения

Предположим, теперь у нас две формы. Но пока они никак не взаимодействуют и ничего друг про друга не знают. Объявим в коде первой формы вторую форму и дадим ей имя:

```
public partial class Form1 : Form
{
    public Form2 frm2;    //Объявляем Form2 и даем ей имя frm2
```

Так же поступаем со второй формой:

```
public partial class Form2 : Form
{
    public Form1 frm1;    //Объявляем Form1 и даем ей имя frm1
```

Многооконные приложения

Положим на обе формы кнопку (button) и метку (label).

По нашему замыслу мы нажатием кнопки в первой форме будем менять текст в метке второй формы и наоборот. Чтобы это стало возможным, мы должны у обеих меток установить свойство **Modifiers public** (по умолчанию – private).

Дальнейшие действия представлены в обработчиках кнопок первой и второй формы и комментариях к ним.

// Первая форма:

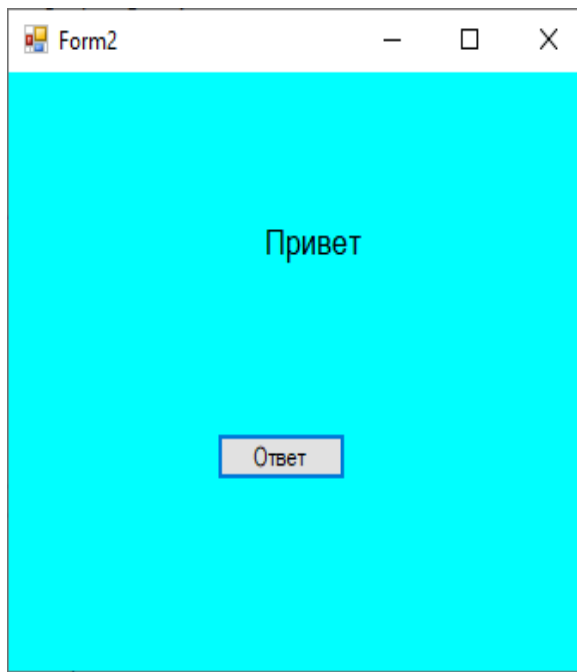
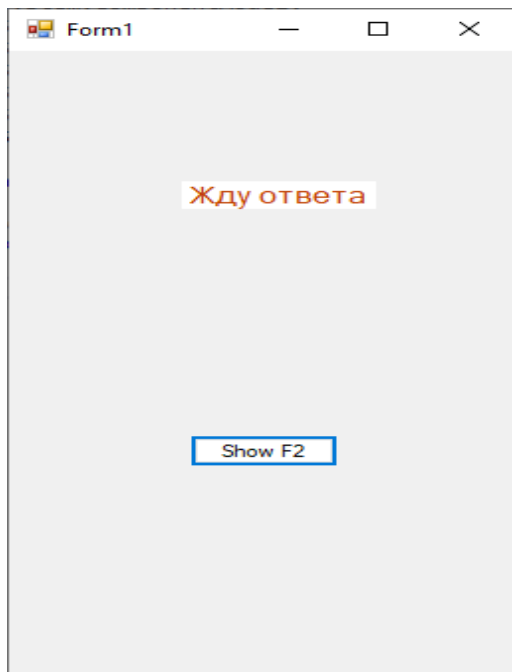
```
private void button1_Click(object sender, EventArgs e)
{
    frm2 = new Form2();//Создаем указатель на форму2
    frm2.frm1 = this;//Передаем форме2 указатель на форму1
    frm2.BackColor = Color.Cyan;//Меняем цвет формы2
    frm2.Show();//Показываем форму2
    frm2.label1.Text = "Привет";//Меняем текст
    this.Hide();//Прячем форму1
}
```

// Вторая форма:

```
private void button1_Click(object sender, EventArgs e)
{
    frm1 = new Form1();
    frm1.label1.Text = "Ответ";
    this.Hide();//Прячем форму2
    frm1.Show();//Показываем форму1
}
```

Многооконные приложения

На рисунке показаны первая форма, вторая форма и первая форма после взаимодействия со второй.



Важно: в определении функций и переменных, к которым мы хотим получить доступ, должен присутствовать модификатор доступа ***public***, а для элементов управления свойство ***Modifiers*** должно быть установлено в ***Public***.

Задание 1

- Создать две взаимодействующие формы, как в примере

Контейнеры в Windows Forms.

Контейнер GroupBox

Назначение контейнера понятно из его названия – хранение и группировка компонентов формы.

GroupBox представляет собой специальный контейнер, который ограничен от остальной формы границей. Он имеет заголовок, который устанавливается через свойство **Text**. Нередко этот элемент используется для группирования переключателей - элементов **RadioButton**, так как позволяет разграничить их группы

Свойство	Описание
Text	Заголовок — текст, поясняющий назначение компонентов, которые находятся в поле компонента GroupBox
Enabled	Позволяет управлять доступом к компонентам, находящимся в поле (на поверхности) компонента GroupBox. Если значение свойства равно false, то все находящиеся в поле GroupBox компоненты недоступны
Visible	Позволяет скрыть (сделать невидимым) компонент GroupBox и все компоненты, которые находятся на его поверхности

Контейнеры в Windows Forms.

Контейнер Panel

Компонент **Panel** представляет собой контейнер для других компонентов и позволяет легко управлять компонентами, которые находятся на панели. Свойства компонента **Panel** приведены в таблице:

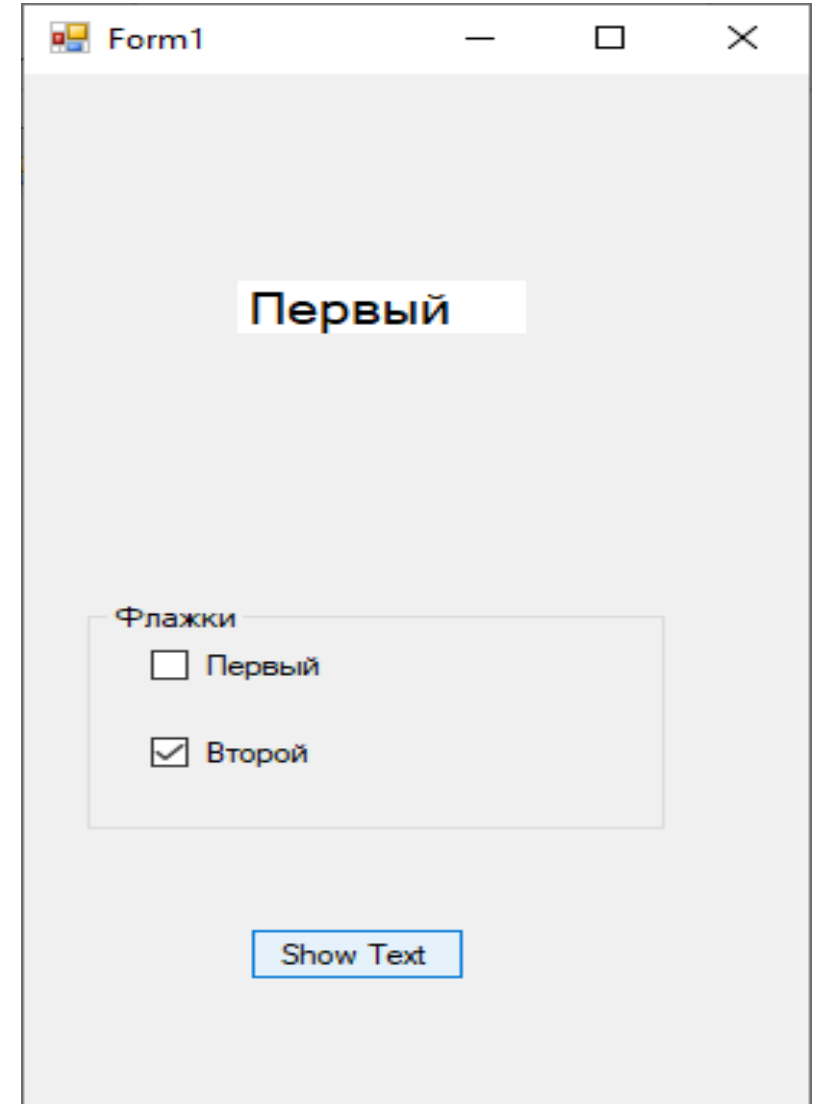
Свойство	Описание
Dock	Определяет границу формы, к которой привязана (прикреплена) панель. Панель может быть прикреплена к левой (Left), правой (Right), верхней (Top) или нижней (Bottom) границе формы
BorderStyle	Вид границы панели: FixedSingle — рамка; Fixed3D — объемная граница; None — граница не отображается
Enabled	Свойство позволяет сделать недоступными (false) все компоненты, которые находятся на панели
Visible	Позволяет скрыть (false) панель

Переключатель CheckBox

Переключатели в некотором смысле являются визуальными аналогами операторов **if** и **switch**.

Компонент **CheckBox** представляет собой переключатель, который может находиться в одном из двух состояний: выбранном или невыбранном.

Приведем пример. В представленном на рисунке окне вы видите два переключателя, один из которых выбран. Кнопка меняет выбор переключателя и отображает текст выбранного переключателя в **Label**. Флажки помещены в контейнер **GroupBox**.



Свойства компонента CheckBox.

Свойство	Значение
Text	Текст, отображаемый справа от кнопки Checked
Checked	Состояние переключателя. Если переключатель выбран (в поле компонента отображается галочка), то значение свойства равно true . Если переключатель сброшен (галочка не отображается), то значение свойства равно false
TextAlign	Положение текста в поле отображения текста. Текст может располагаться в центре поля (MiddleCenter), быть прижат к левой (MiddleLeft) или правой (MiddleRight) границе. Можно задать и другие способы размещения текста надписи (TopLeft, TopCenter, TopRight, BottomLeft, BottomCenter, BottomRight)
CheckAlign	Положение кнопки в поле компонента. Кнопка может быть прижата к левой верхней границе (TopLeft), прижата к левой границе и находиться на равном расстоянии от верхней и нижней границ поля компонента (MiddleLeft). Есть и другие варианты размещения кнопки в поле компонента
Enabled	Управляет доступностью компонента. Позволяет сделать переключатель недоступным (false)
Visible	Управляет видимостью компонента. Позволяет скрыть, сделать невидимым (false) переключатель

Свойства компонента CheckBox. Продолжение.

Свойство	Значение
AutoCheck	Определяет, должно ли автоматически изменяться состояние переключателя в результате щелчка на его изображении. По умолчанию значение равно true
FlatStyle	Стиль (вид) переключателя. Переключатель может быть обычным (Standard), плоским (Flat) или "всплывающим" (Popup). Стиль определяет поведение переключателя при позиционировании указателя мыши на его изображении
Appearance	Определяет вид переключателя. Переключатель может выглядеть обычным образом (Normal) или как кнопка (Button)
Image	Картинка, которая отображается в поле компонента Положение картинки в поле компонента. Картинка может располагаться в центре (MiddleCenter), быть прижата к левой (MiddleLeft) или правой (MiddleRight) границе. Можно задать и другие способы размещения картинки на кнопке (TopLeft, TopCenter, TopRight, BottomLeft, BottomCenter, BottomRight)
ImageAlign	Положение картинки в поле компонента. Картинка может располагаться в центре (MiddleCenter), быть прижата к левой (MiddleLeft) или правой (MiddleRight) границе. Можно задать и другие способы размещения картинки на кнопке (TopLeft, TopCenter, TopRight, BottomLeft, BottomCenter, BottomRight)
ImageList	Набор картинок, используемых для обозначения различных состояний кнопки. Представляет собой объект типа ImageList . Чтобы задать значение свойства, в форму приложения следует добавить компонент ImageList
ImageIndex	Номер (индекс) картинки из набора ImageList, которая отображается в поле компонента

Переключатель CheckBox

```
public Form1()  
{  
    InitializeComponent();  
}  
private void button1_Click(object sender, EventArgs e)  
{  
    if (checkBox1.Checked)  
    { label1.Text = checkBox1.Text;  
      checkBox1.Checked = false;  
      checkBox2.Checked = true;  
    }  
    else  
    {  
        label1.Text = checkBox2.Text;  
        checkBox2.Checked = false;  
        checkBox1.Checked = true;  
    }  
}
```

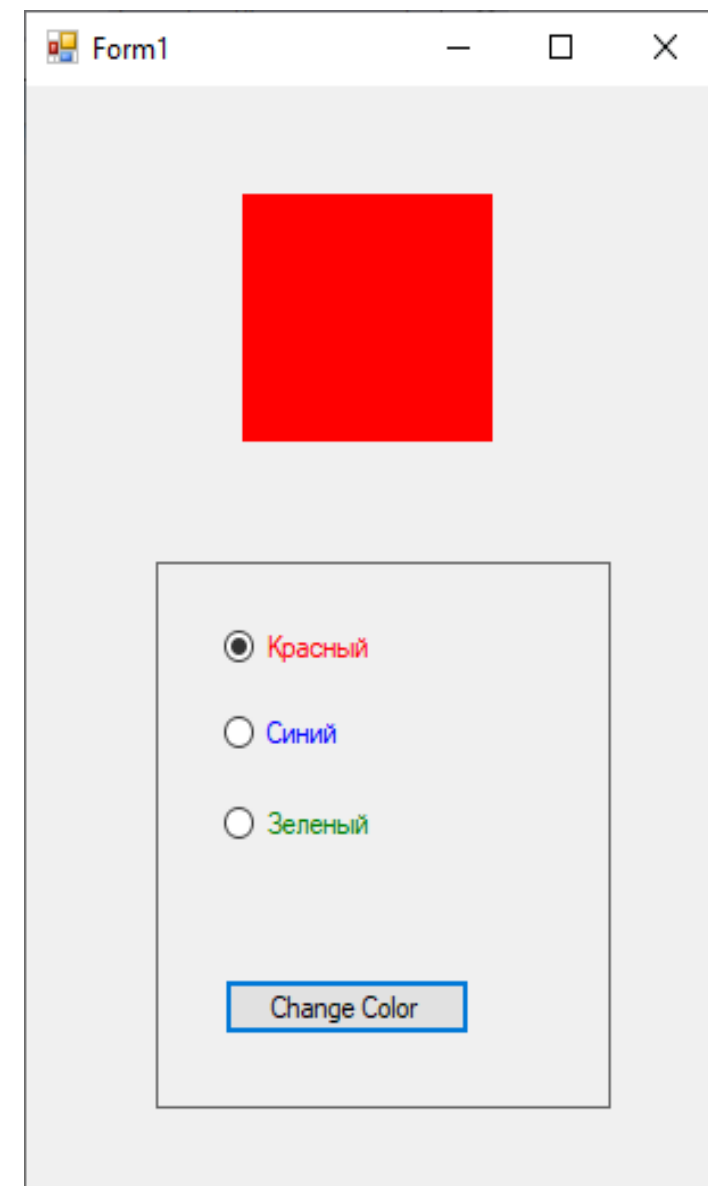
Задание 2

- Сделать переключатель **CheckBox** как в примере

Переключатель *RadioButton*

Радиокнопка это зависимый переключатель, это означает, что из нескольких радиокнопок во включенном состоянии может находиться только одна кнопка. Радиокнопки, помещенные в контейнер работают, как группа зависимых переключателей, т.е. во включенном состоянии может находиться только одна кнопка. Контейнером для радиокнопок может служить форма, элементы **GroupBox** и **Panel**. Основные свойства **RadioButton** идентичны свойствам **CheckBox**, и вы можете видеть их в табл.12.

На рисунке представлена форма с тремя переключателями **RadioButton**. В зависимости от их выбора, кнопка переключает цвет метки.



Переключатель RadioButton

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
    }

    private void button1_Click(object sender, EventArgs e)
    {
        if (radioButton1.Checked) label1.BackColor = Color.Red;
        if (radioButton2.Checked) label1.BackColor = Color.Blue;
        if (radioButton3.Checked) label1.BackColor = Color.Green;
    }
}
```

Задание 3

- Сделать контейнер Panel или GroupBox и переключатели Radiobutton как в примере

Окно сообщений MessageBox

Как правило, для вывода сообщений применяется элемент MessageBox. Однако кроме вывода строки сообщения данный элемент может устанавливать ряд настроек, которые определяют его поведение. Для вывода сообщения в классе MessageBox предусмотрен метод Show, который имеет различные версии и может принимать ряд параметров. Упрощенный синтаксис этого метода приведен ниже:

Show(<текст в окне>[,<заголовок окна>,<набор кнопок>,<иконка>,<выбранная по умолчанию кнопка >])

Рассмотрим более подробно параметры метода Show. Все параметры, кроме первого, являются необязательными. Первый и второй параметры метода строкового типа. Остальные параметры с помощью набора констант задают кнопки, иконки в окне и кнопку по умолчанию.

Окно сообщений MessageBox

Тип **MessageBoxButtons** задает набор кнопок в окне.

Набор констант типа **MessageBoxButtons**, приведен в таблице:

Значение	Описание
AbortRetryIgnore	Окно содержит кнопки Abort, Retry и Ignore.
OK	Окно содержит кнопку OK.
OKCancel	Окно содержит кнопки OK и Cancel.
RetryCancel	Окно содержит кнопки Retry и Cancel.
YesNo	Окно содержит кнопки Yes и No.
YesNoCancel	Окно содержит кнопки Yes, No, и Cancel.

Окно сообщений *MessageBox*

Тип `MessageBoxIcon` задает иконку в окне. Набор констант типа `MessageBoxIcon`, приведен в таблице:

Значение	Описание
Asterisk	Сообщение содержит символ, состоящий из маленькой буквы i в кружке
Error	Пиктограмма ошибочного сообщения
Exclamation	Пиктограмма содержит восклицательный знак
Hand	Пиктограмма содержит символ, состоящий из белого крестика в кружке на красном фоне.
Information	Информационная пиктограмма
None	Ничего не содержит.
Question	Пиктограмма содержит вопросительный знак
Stop	Пиктограмма содержит символ, состоящий из белого крестика в кружке на красном фоне.
Warning	Пиктограмма содержит символ, состоящий из восклицательного знака в треугольнике на желтом фоне.

Окно сообщений *MessageBox*

Перечисление `MessageBoxDefaultButton` определяет кнопку, выбранную по умолчанию. В таблице приведены члены перечисления:

Значение	Описание
Button1	Первая кнопка в окне сообщения является кнопкой по умолчанию
Button2	Вторая кнопка в окне сообщения является кнопкой по умолчанию
Button3	Третья кнопка в окне сообщения является кнопкой по умолчанию

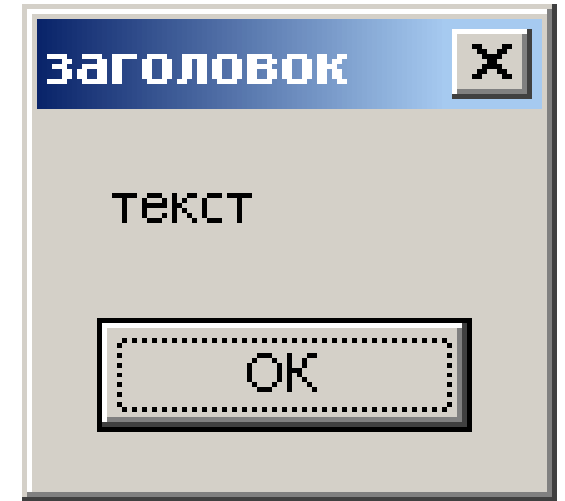
Метод `Show` возвращает значение типа `DialogResult`, который предоставляет набор констант, сигнализирующих какая из кнопок выбрана пользователем:

Значение	Описание
Abort	Нажата кнопка <code>Abort</code> .
Cancel	Нажата кнопка <code>Cancel</code> .
Ignore	Нажата кнопка <code>Ignore</code> .
No	Нажата кнопка <code>No</code> .
None	Ничего не нажато.
OK	Нажата кнопка <code>OK</code> .
Retry	Нажата кнопка <code>Retry</code> .
Yes	Нажата кнопка <code>Yes</code> .

Примеры использования класса MessageBox.

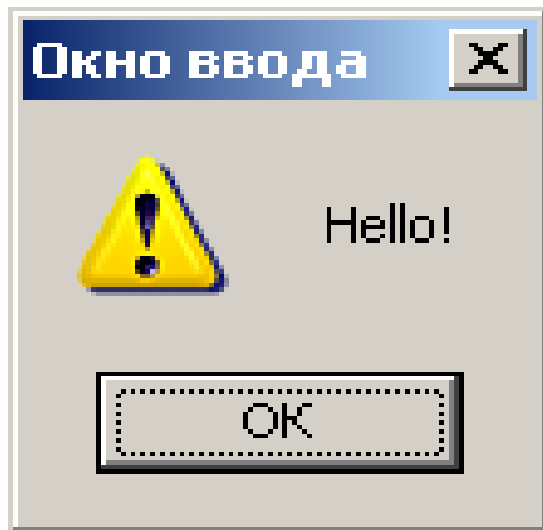
1. Выдача сообщения в окне.

```
MessageBox.Show("Текст", "Заголовок окна");
```



2. Использование иконки в окне:

```
MessageBox.Show("Hello!", "Окно ввода", MessageBoxButtons.OK,  
MessageBoxIcon.Exclamation);
```



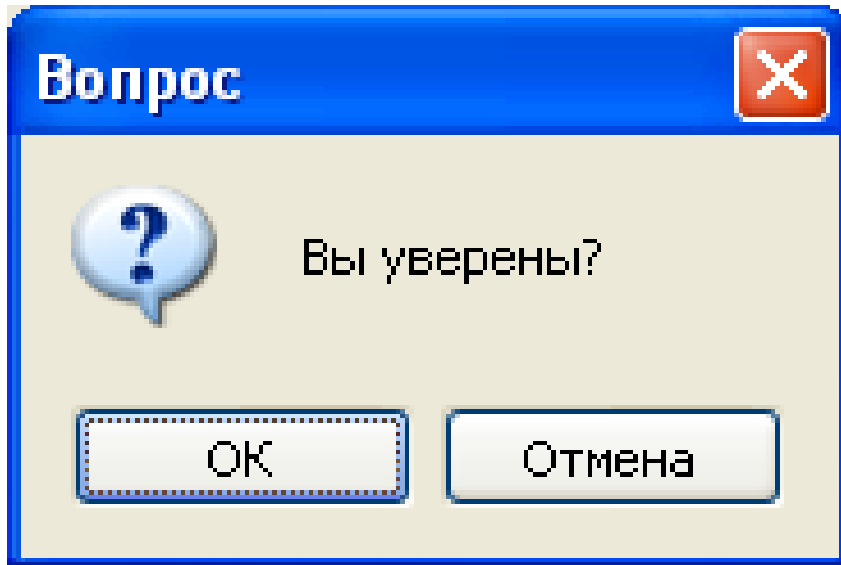
Задание 4

- Запустить тестовый пример с MessageBox как в самом первом примере

Примеры использования класса MessageBox.

3. Использование нескольких кнопок // удалить текущий элемент

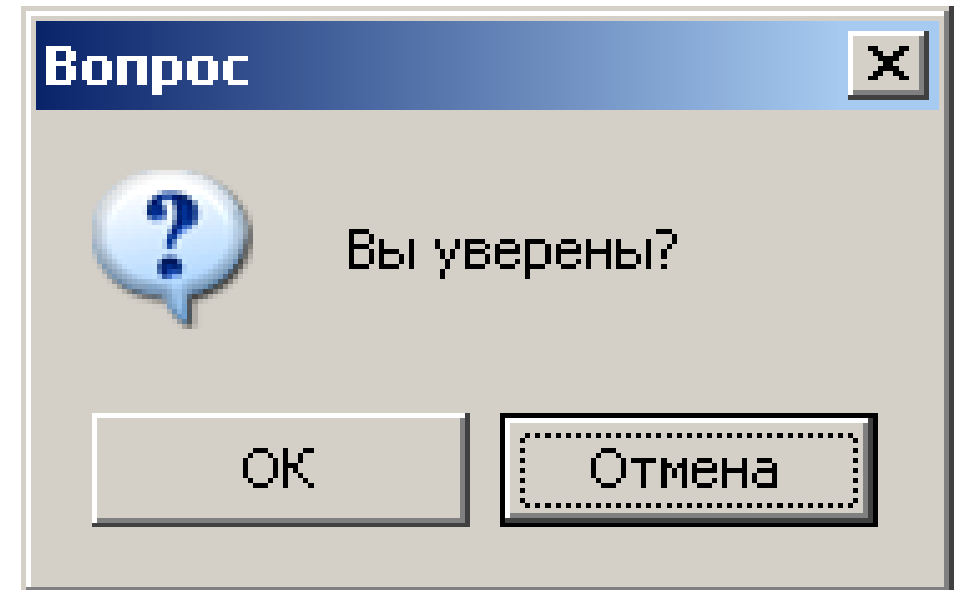
```
if (MessageBox.Show("Вы уверены?", "Вопрос",  
    MessageBoxButtons.OKCancel, MessageBoxIcon.Question) ==  
    DialogResult.OK)  
    listBox1.Items.RemoveAt(listBox1.SelectedItem);
```



Примеры использования класса MessageBox.

4. Кнопка «Отмена», выбранная по умолчанию:

```
if (MessageBox.Show("Вы уверены?", "Вопрос",  
    MessageBoxButtons.OKCancel, MessageBoxIcon.Question,  
    MessageBoxDefaultButton.Button2) == DialogResult.OK)  
{  
    MessageBox.Show("ok");  
}
```



Диалоги открытия и сохранения файлов

Окна открытия и сохранения файла представлены классами **OpenFileDialog** и **SaveFileDialog**. Элемент **OpenFileDialog** находится на вкладке «Диалоговые окна» панели инструментов и позволяет для выбора использовать стандартный **Windows-диалог**. Этот компонент является «невидимым». Его вызов осуществляется программным путем.

Свойство	Описание
Title	Текст в заголовке окна. Если значение свойства не указано, то в заголовке отображается текст Открыть
Filter	Свойство задает описание и фильтр (маску) имени файла. В списке файлов отображаются только те файлы, имена которых соответствуют указанной маске. Описание отображается в поле Тип файла . Например, значение Текст *.txt указывает, что в списке файлов надо отобразить только те файлы, расширение которых txt
FilterIndex	Фильтр состоит из нескольких элементов (например, Текст *.txt Все файлы *.*) , то значение свойства задает фильтр, который используется в момент появления диалога на экране
FileName	Имя выбранного пользователем файла
InitialDirectory	Каталог, содержимое которого отображается при появлении диалога на экране
RestoreDirectory	Признак необходимости отображать содержимое каталога, указанного в свойстве InitialDirectory , при каждом появлении окна. Если значение свойства равно False , то при следующем появлении окна отображается содержимое каталога

Диалоги открытия и сохранения файлов

Элемент **SaveFileDialog** можно найти на вкладке «Диалоговые окна». **OpenFileDialog** и **SaveFileDialog** имеют ряд общих свойств. Они приведены в таблице на предыдущем слайде. В следующей таблице приведены свойства, специфические для **SaveFileDialog**:

Свойство	Описание
CheckPathExists	Признак необходимости проверки существования каталога, в котором следует сохранить файл. Если указанного каталога нет, то выводится информационное сообщение
CheckFileExists	Признак необходимости проверки существования файла с заданным именем. Если значение свойства равно True и файл с указанным именем уже существует, то появляется окно запроса, в котором пользователь может подтвердить необходимость замены (перезаписи) существующего файла

Пример чтения из файла в TextBox с использованием OpenFileDialog.

```
private void button1_Click(object sender, EventArgs e)
{
    //Чтение из файла в TextBox
    if (openFileDialog1.ShowDialog() == DialogResult.OK)
    {
        string filename = openFileDialog1.FileName;
        string fileText = File.ReadAllText(filename);
        textBox1.Text = fileText;
    }
}
```

Пример записи из TextBox в файл с использованием SaveFileDialog.

```
private void button2_Click(object sender, EventArgs e)
{
    //Запись из TextBox в файл
    saveFileDialog1.Filter = "Text files(*.txt)|*.txt|All
                             files(*.*)|*.*";
    if (saveFileDialog1.ShowDialog() == DialogResult.Cancel)
        return;
    string filename = saveFileDialog1.FileName;
    File.WriteAllText(filename, textBox1.Text);
}
```

Компонент *ListBox*

Компонент **ListBox** представляет собой список, в котором можно выбрать нужный элемент. Ключевым свойством этого элемента является свойство **Items**, которое как раз и хранит набор всех элементов списка. Основные свойства компонента приведены в таблице:

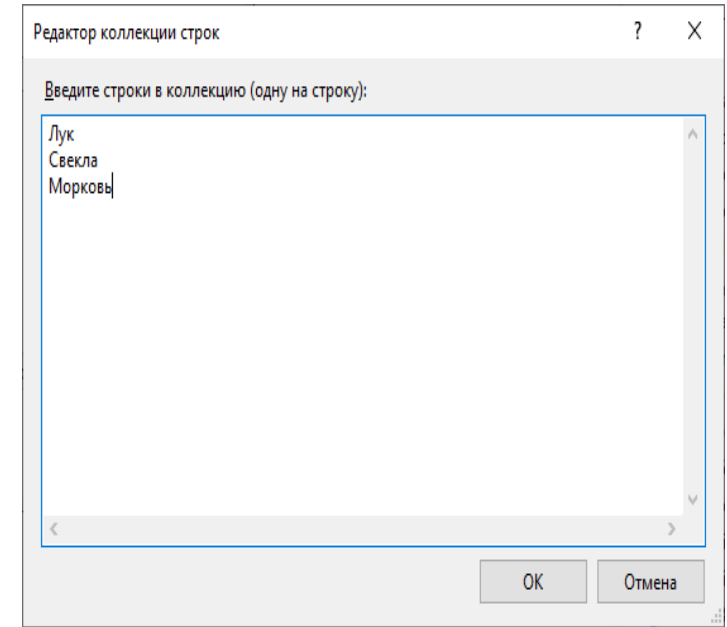
Свойство	Описание
Items	Элементы списка — коллекция строк
Items.Count	Количество элементов списка
SelectedIndex	Номер элемента, выбранного в списке. Если ни один из элементов списка не выбран, то значение свойства равно -1
Sorted	Признак необходимости автоматической сортировки (True) списка после добавления очередного элемента
SelectionMode	Определяет режим выбора элементов списка: One — только один элемент; MultiSimple — можно выбрать несколько элементов, сделав щелчок на нужных элементах списка; MultiExtended — можно выбрать несколько элементов, сделав щелчок на нужных элементах списка при нажатой клавише <Ctrl> , или выделить диапазон, щелкнув при нажатой клавише <Shift> на первом и последнем элементе диапазона

Задание 5

- Создать как в примере чтение из файла и запись в файл, добавив на форму две компоненты (разные) работы с файл-диалог.
- В using добавить System.IO

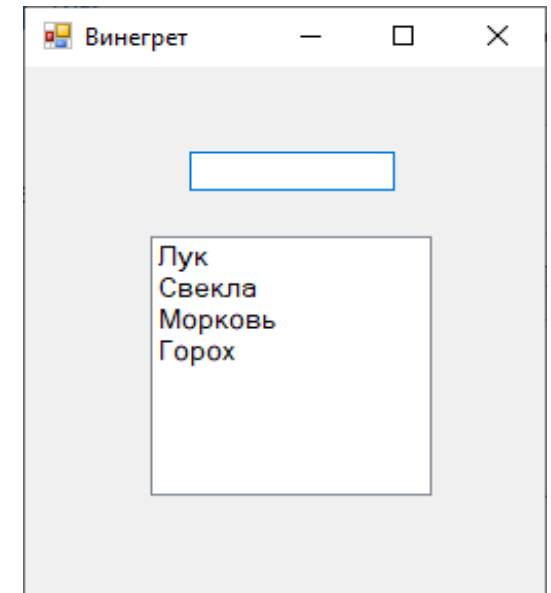
Компонент ListBox

Элементы в список могут добавляться как во время разработки, так и программным способом. В **Visual Studio** в окне **Properties (Свойства)** для элемента **ListBox** мы можем найти свойство **Items**. После двойного щелчка на свойство нам отобразится окно для добавления элементов в список.



Окно добавления элементов

В пустое поле мы вводим по одному элементу списка - по одному на каждой строке. После этого все добавленные нами элементы окажутся в списке, и мы **сможем** ими управлять:



ListBox с введенными элементами

Компонент *ListBox*

Все элементы списка входят в свойство **Items**, которое представляет собой коллекцию. Для добавления нового элемента в эту коллекцию, а значит и в список, надо использовать метод **Add**, например: **listBox1.Items.Add("Новый элемент");**. При использовании этого метода каждый добавляемый элемент добавляется в конец списка.

Можно добавить сразу несколько элементов, например, массив. Для этого используется метод **AddRange**:

```
string[] countries = { "Бразилия", "Аргентина", "Чили", "Уругвай",  
"Колумбия"};
```

```
listBox1.Items.AddRange(countries)
```

В отличие от простого добавления вставка производится по определенному индексу списка с помощью метода **Insert**. В данном случае вставляем элемент на вторую позицию в списке, так как отсчет позиций начинается с нуля.

```
listBox1.Items.Insert(1, "Парагвай");
```

*Компонент **ListBox***

Для удаления элемента по его тексту используется метод **Remove**:
listBox1.Items.Remove("Чили");

Чтобы удалить элемент по его индексу в списке, используется метод **RemoveAt**:
listBox1.Items.RemoveAt(1);

Кроме того, можно очистить сразу весь список, применив метод **Clear**:
listBox1.Items.Clear();

Используя **индекс элемента**, можно найти сам элемент в списке. Например, получим первый элемент списка:

```
string firstElement = listBox1.Items[0];
```

Метод **Count** позволяет определить количество элементов в списке.

Компонент ListBox

При выделении элементов списка мы можем ими управлять как через индекс, так и через сам выделенный элемент. Получить выделенные элементы можно с помощью следующих свойств элемента **ListBox**:

SelectedIndex: возвращает или устанавливает номер выделенного элемента списка. Если выделенные элементы отсутствуют, тогда свойство имеет значение **-1**

SelectedIndices: возвращает или устанавливает коллекцию выделенных элементов в виде набора их индексов

SelectedItem: возвращает или устанавливает текст выделенного элемента

SelectedItems: возвращает или устанавливает выделенные элементы в виде коллекции

Компонент ListBox

По умолчанию список поддерживает выделение одного элемента.

Чтобы добавить возможность выделения нескольких элементов, надо установить у его свойства **SelectionMode** значение **MultiSimple**.

Чтобы выделить элемент программно, надо применить метод **SetSelected(int index, bool value)**,
где **index** - номер выделенного элемента.

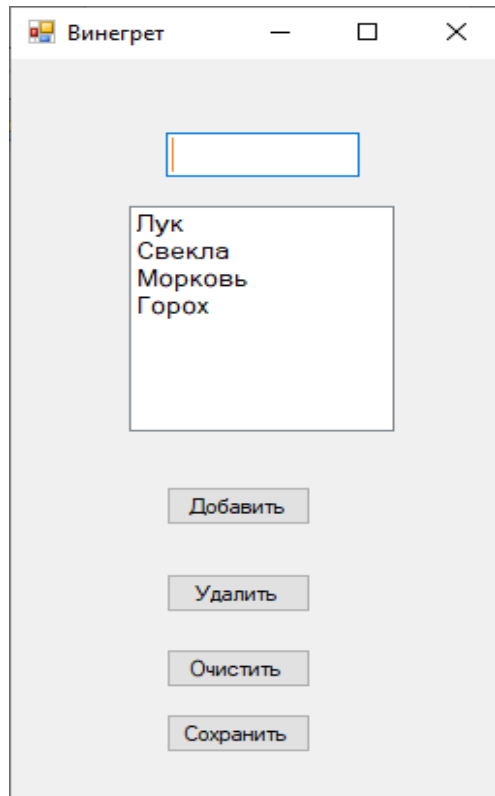
Если второй параметр - **value** имеет значени **true**, то элемент по указанному индексу выделяется, если **false**, то выделение, наоборот, скрывается:

listBox1.SetSelected(2, true); // **будет выделен третий элемент**

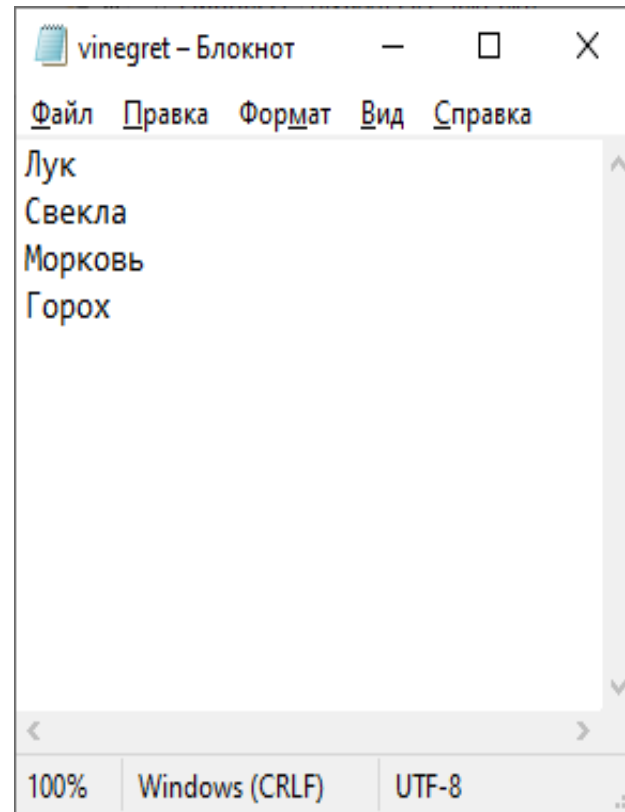
Чтобы снять выделение со всех выделенных элементов, используется метод **ClearSelected**.

*Пример использования компонента **ListBox***

На рисунке представлена форма со списком **ListBox** и текстовый файл, в котором он сохранен:



ListBox



текстовый файл.

Код программы работы со списком ListBox для этого примера.

```
private void button4_Click(object sender, EventArgs e)
{
    //Сохранение в файле
    saveFileDialog1.Filter = "Text files(*.txt)|*.txt|All files(*.*)|*.*";
    if (saveFileDialog1.ShowDialog() == DialogResult.Cancel)
        return;
    string filename = saveFileDialog1.FileName;
    WriteToFile(filename, listBox1);
}

private void WriteToFile(string path, ListBox listBox)
{
    using (var sw = new StreamWriter(new FileStream(path, FileMode.Create)))
    {
        if (listBox != null)
        {
            foreach (var item in listBox.Items) // в таком же порядке
            {
                sw.WriteLine(item.ToString());
            }
        }
    }
}
```

Код программы работы со списком ListBox для этого примера. Продолжение.

```
private void button1_Click(object sender, EventArgs e)
{
    //Добавляем элемент
    listBox1.Items.Add(textBox1.Text);
}
private void button2_Click(object sender, EventArgs e)
{
    //Удаляем элемент
    listBox1.Items.Remove("Горox");
}
private void button3_Click(object sender, EventArgs e)
{
    //Очищаем список
    listBox1.Items.Clear();
}
}
```

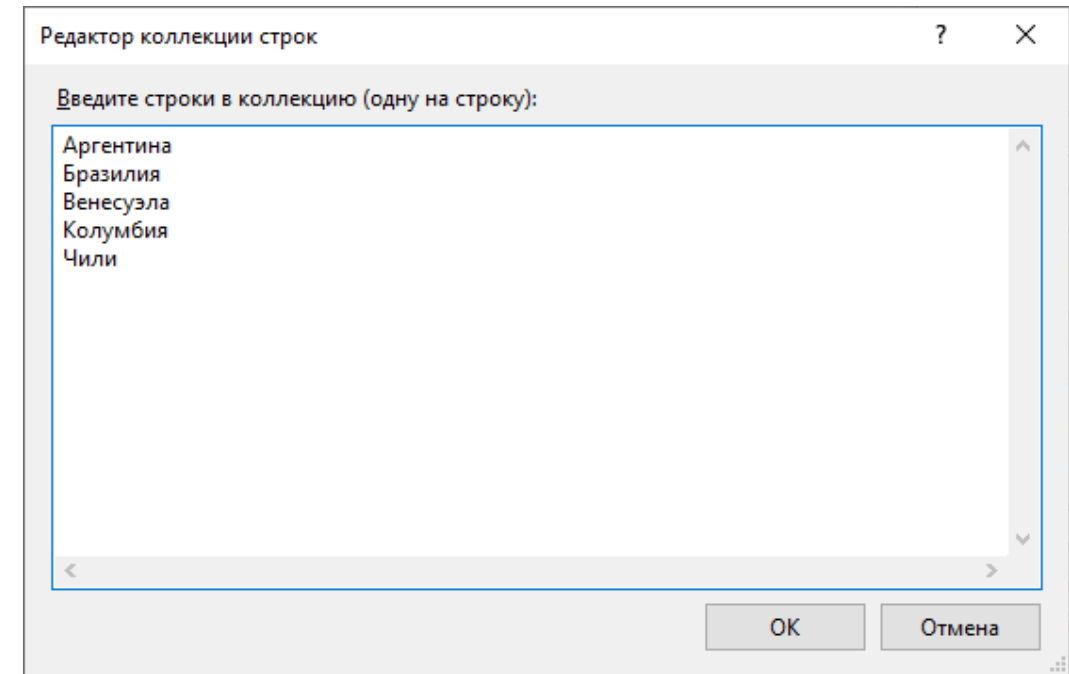
Задание 6 для самостоятельной работы

Напишите программу, состоящую из двух списков **Listbox**. В первом будет, например, перечень товаров, заданный программно. Второй изначально пуст, пусть это будет перечень покупок. При клике на одну кнопку товар должен переходить из одного списка в другой. При клике на вторую кнопку – возвращаться (человек передумал покупать). (**Предусмотрите возможность множественного выбора элементов списка и их перемещения- задание повышенной сложности**).

Список **ComboBox**.

Компонент **ComboBox** совмещает в себе список и текстовое поле. По этой причине у него есть свойство **Text** – текст, находящийся в поле ввода/редактирования. Отметим также свойство **DropDownStyle** – вид компонента: **DropDown** – поле ввода и раскрывающийся список; **Simple** – поле ввода со списком; **DropDownList** – раскрывающийся список. Ниже мы поясним действие этого свойства на примерах. Остальные его свойства аналогичны свойствам **ListBox** (см. таблицу свойств **ListBox**).

Для хранения элементов списка в **ComboBox** также предназначено свойство **Items**. Подобным образом, как и с **ListBox**, мы можем в окне свойств выбрать свойство **Items** и нам отобразится окно для добавления элементов **ComboBox**:



Редактор строк **ComboBox**.

Список ComboBox.

И, как и с компонентом ListBox, здесь мы также можем программно управлять элементами.

Добавление элементов:

```
comboBox1.Items.Add("Парагвай");// добавляем один элемент  
comboBox1.Items.AddRange(new string[] { "Уругвай", "Эквадор" });  
                                     //добавляем набор элементов  
comboBox1.Items.Insert(1, "Боливия");// добавляем один элемент  
                                     // на определенную позицию
```

Удаление элементов:

```
comboBox1.Items.Remove("Аргентина");// удаляем один элемент  
comboBox1.Items.RemoveAt(1); // удаляем элемент по индексу  
comboBox1.Items.Clear(); // удаляем все элементы
```

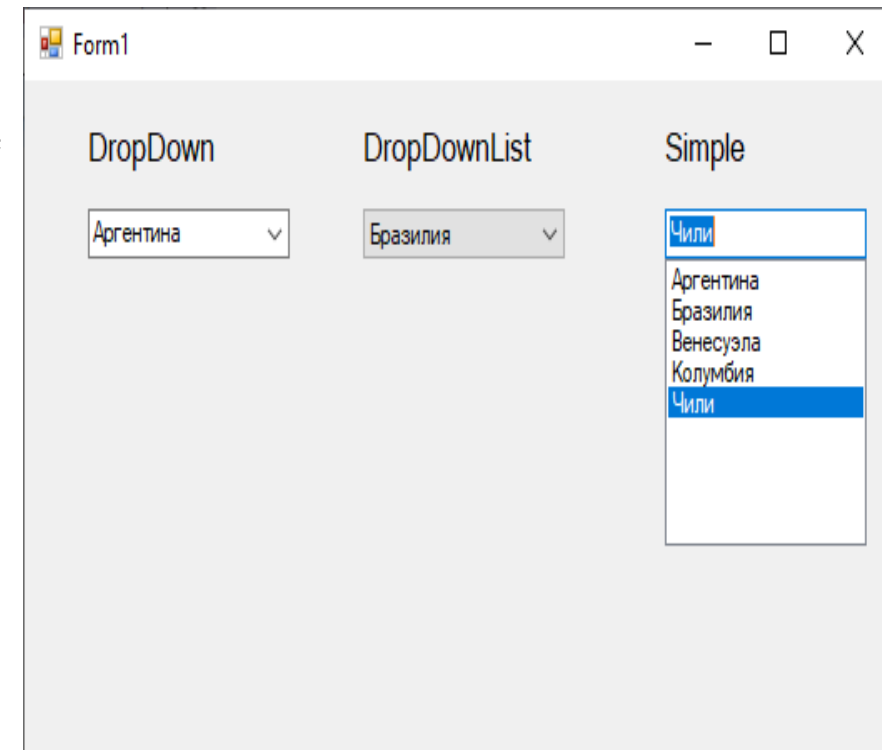
Мы можем получить элемент по индексу и производить с ним разные действия.
Например, изменить его

```
comboBox1.Items[0] = "Парагвай";
```

Свойства ComboBox.

С помощью ряда свойств можно настроить стиль оформления компонента:

- ❖ Свойство **DropDownWidth** и **DropDownHeight** задает ширину и высоту выпадающего списка.
- ❖ Свойство **MaxDropDownItems** позволяет задать число видимых элементов списка — от 1 до 100. (По умолчанию это число равно 8.)
- ❖ Свойство **DropDownStyle** задает стиль **ComboBox**. Оно может принимать три возможных значения:
 - ❖ **DropDown**: используется по умолчанию. Мы можем открыть выпадающий список вариантов при вводе значения в текстовое поле или нажав на кнопку со стрелкой в правой части элемента, и нам отобразится собственно выпадающий список, в котором можно выбрать возможный вариант
 - ❖ **DropDownList**: чтобы открыть выпадающий список, надо нажать на кнопку со стрелкой в правой стороне элемента
 - ❖ **Simple**: ComboBox представляет простое текстовое поле, в котором для перехода между элементами мы можем использовать клавиши клавиатуры вверх/вниз



Задание 7

Form1

Наименование товара	Цена за единицу товара	Количество единиц товара в покупке
Простой карандаш	10	0
Шариковая ручка	20	1
Стирательная резинка	15	2
Тетрадь 48 л	50	3
Тетрадь 96 л	100	0
Тетрадь 24 л	20	0

Изменить

Вычислить стоимость

95

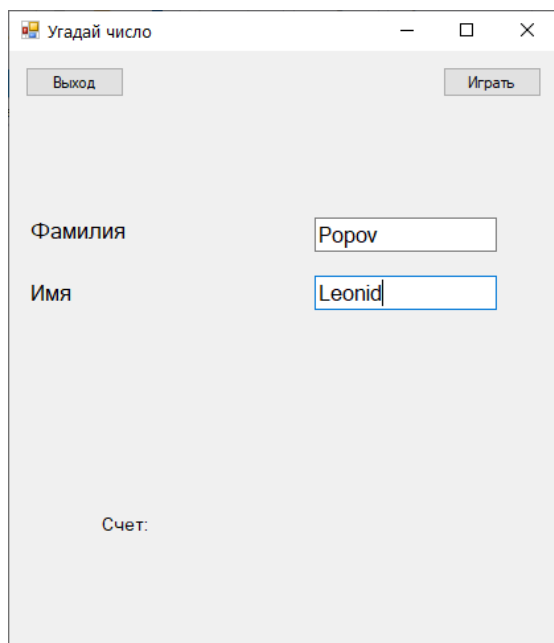
Задание 7

Задание для самостоятельной работы

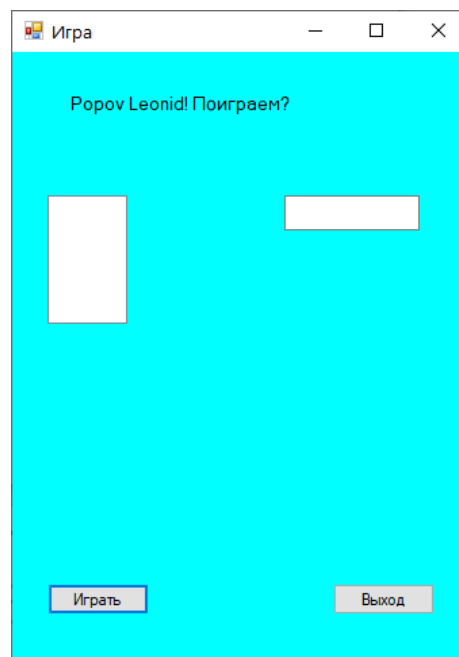
- **Задача 1. Покупатель.** Напишите программу, состоящую из двух списков *Listbox*. В первом будет, например, перечень товаров, заданный программно. Вторым изначально пуст, пусть это будет перечень покупок. При клике на одну кнопку товар должен переходить из одного списка в другой. При клике на вторую кнопку – возвращаться (человек передумал покупать). Предусмотрите возможность множественного выбора элементов списка и их перемещения.
- **Задача 2. Магазин.** Есть список из 10 товаров, например, канцелярских, и соответствующий им список цен. Надо в 3-м списке указать количество каждого товара, которые необходимо купить, например для класса или группы в детсад. При этом нельзя задавать одинаковое количество для всех наименований. Например, надо купить простых карандашей 40 штук, фломастеров – 6 коробок, альбомов – 20, ластиков – 15 и т.п. Цены разумные! Программа должна подсчитать, сколько рублей потребуется. Подсчет должен идти в цикле.

Учебная программа «Угадай число»

Суть игры проста: игрок загадывает и вводит цифру, а программа генерирует случайное число. Если эти числа совпали, игрок получает бонусы. Код программы с комментариями приведен ниже.



The screenshot shows a window titled "Угадай число" (Guess the number). It has a light gray background. At the top left is a "Выход" (Exit) button, and at the top right is an "Играть" (Play) button. Below these, there are two input fields. The first is labeled "Фамилия" (Surname) and contains the text "Popov". The second is labeled "Имя" (Name) and contains the text "Leonid". At the bottom left, there is a label "Счет:" (Score:).



The screenshot shows a window titled "Игра" (Game). It has a bright cyan background. At the top, it says "Popov Leonid! Поиграем?" (Popov Leonid! Let's play?). Below this, there are two empty input fields. At the bottom, there are two buttons: "Играть" (Play) on the left and "Выход" (Exit) on the right.

Программа «Угадай число»

```
namespace Ugadai_chislo
{
    public partial class Form1 : Form
    {
        public Form2 frm2;
        public Form1()
        {
            InitializeComponent();
        }
        private void button1_Click (object sender, EventArgs e)
        {
            this.Close();//Конец игры
        }
    }
}
```

Программа «Угадай число»

```
private void button2_Click(object sender, EventArgs e)
{
    frm2 = new Form2();//Создаем указатель на форму2
    frm2.frm1 = this;//Передаем форме2 указатель на форму1
    frm2.BackColor = Color.Сyan;//Меняем цвет формы2
    frm2.Show();//Показываем форму2
    frm2.label1.Text = textBox1.Text + " " + textBox2.Text +
        "! Поиграем?";
}
}
```

Программа «Угадай число»

```
namespace Ugadai_chislo
{
    public partial class Form2 : Form
    {
        public Form1 frm1;
        int c = 0;
        int k = 0;
        Random rnd = new Random();
        public Form2()
        {
            InitializeComponent();
        }
    }
}
```

Программа «Угадай число»

```
private void button1_Click(object sender, EventArgs e)
{
    int b, d=0;
    bool f = true;
    string s = textBox1.Text; // считываем число игрока
    try
    { //Проверка: вдруг не число
        d = Convert.ToInt32(s);
        c++; //счет ходов
    }
    catch
    {
        textBox1.Text = "";
        f = false;
    }
}
```

Программа «Угадай число»

```
if (f)
{
    //Если число
    b = rnd.Next(1,6);
    if (d==b)
    {
        k++; //Счет совпадений
    }
    s = s + " - " + b.ToString();
    listBox1.Items.Add(s);
}
if (c==5) //Всего пять ходов
{
    textBox1.Enabled = false;
    button1.Enabled = false;
}
}
```

Программа «Угадай число»

[illegible]

Задание 8

- Сделать по примеру задачу «Угадай число».
- Самостоятельно описать необходимые переменные